

Replit: Kontrollfrågor & Verifiering av Traivo Databasstruktur

Datum: 2026-06-11

Syfte: Säkerställa att Replit och Mats har EXAKT samma mentala modell av databasen

Prioritet: KRITISK – detta måste verifieras INNAN vidare implementation

A. Sammanfattning av Mats logiska tänk

Mats beskriver en databasstruktur för Traivo (fältservice-planering) som är uppbyggd kring **sex huvudregister** och **fyra-fem sidoregister**. Den centrala designprincipen är att **metadata är det universella beskrivningsspråket** – objekt definieras INTE genom typerregister utan genom flexibla metadatafält.

Systemets motor är **orderkonceptet**, som automatiskt genererar uppgifter genom att matcha artiklarnas metadataavillkor mot objektens metadata. Kvantiteter löses i tre steg (orderkoncept → objektmetadata → utförare). Fortnox är master för kund- och artikelnummer.

Huvudflödet: **Kund → Orderkoncept → Artiklar + Objekt (via metadata-matchning) → Uppgifter → Grovplanering → Utförare → Fakturering**

B. KRITISKA KONTROLLFRÅGOR

INSTRUKTION TILL REPLIT: Svara på VARJE fråga nedan innan du gör NÅGON implementation eller ändring. Markera varje svar med  (stämmer),  (delvis/avvikelse) eller  (stämmer inte).

B1. Förståelseverifiering - Övergripande

Fråga 1: “Här är min förståelse av den övergripande arkitekturen:

Traivo har sex huvudregister:

1. **Kundregister** – synkat med Fortnox, enkel data + associationer
2. **Artikelregister** – synkat med Fortnox, med metadataavillkor för antal, “haka på”, lämna info, utförandety
3. **Objektregister** – hierarkiskt (förälder/barn), all beskrivning via metadatafält
4. **Metadatarregister** – tvådelat: definitioner (struktur/typ/arv) + värden (faktisk data)
5. **Orderkoncept** – skapar uppgifter automatiskt baserat på artiklar + metadata-matchning mot objekt
6. **Uppgiftslista/Grovplanering** – alla genererade uppgifter i en gemensam lista

Plus sidoregister: Utförandety, Utförarregister, Artikeltyp, Metadatatyp, Ikonregister.

Stämmer detta med din förståelse av nuvarande implementation? Vad avviker?”

B2. Förståelseverifiering - Per entitet

Fråga 2 - Kundregistret:

"Jag förstår att Kundregistret har följande ansvar:

- Kundnummer och namn (från Fortnox via API)
- Prislista associerad per kund
- Framtida: fakturavillkor (vecko/månads/delfaktura)
- Fria datafält som kan läggas till efter behov
- **Tre associationer:** (1) Objekt/Kluster, (2) Orderkoncept, (3) Uppgifter
- Flera kunder kan vara kopplade till samma kluster (N:M)

Är detta korrekt? Hur ser din nuvarande Kund-modell ut?"

Fråga 3 - Artikelregistret:

"Jag förstår att Artikelregistret är det mest komplexa registret med följande delar:

- Grunddata: artikelnummer (Fortnox), beskrivning
- Informationsfält: bilder, PDF:er, externa leverantörer
- Pris/utförande: listpris, kostnad, produktionstid, offsettid, leveranstid
- Geografisk koppling: primär (objektets plats) + sekundär (lagerplats för fysiska artiklar)
- Strukturartiklar: huvudartikel → underartiklar (lista)
- **Metadatatvillkor:**
- Antal: hämta från metadatatfält ELLER fast värde, utförare kan justera (konfigurerbart)
- Lämna information: obligatoriska fält för uppgiftsslutförande (foto, referens, status)
- Haka på: villkor för metadata-matchning (=, ≠, innehåller) med möjlighet till flera villkor
- Utförandetyper: vilken typ av utförare som krävs

Stämmer detta? Finns alla dessa koncept i nuvarande implementation?"

Fråga 4 - Metadatatregistret:

"Jag förstår att metadata hanteras i TVÅ separata register:

- **Register 1 (Definition):** Definierar metadatatfältet - typ (numerisk/alfanumerisk/foto/valmeny/fil), arvlogik (neråt i kluster), galleri vs. enkelvärde, hierarkisk gruppering (t.ex. Kontakt → Namn, Titel, Telefon, E-post)
- **Register 2 (Värden):** Innehåller faktisk data kopplat till objekt/artikel
- Varje metadatatfält har ett eget nummer (referensnummer) och namn
- Automatisk loggning vid uppdatering (vem + när)

Har du implementerat detta som två separata tabeller/modeller? Hur hanterar du arv neråt i klustret?"

Fråga 5 - Objektregistret:

"Jag förstår att Objektregistret har:

- Autogenererat löpnummer (primärnyckel)
- Objektamn
- Föräldranummer (hierarkisk trädstruktur/kluster)
- ALL beskrivande data via metadatatfält (INTE separata kolumner)
- Geografisk positionering via metadata: ort → gatuadress → What3Words → lon/lat
- **Systemregel:** Mest högupplöst position används för planering
- **Systemregel:** Objekt inom 30m = ett 'jobb' (konfigurerbar radie)
- **KRITISKT BESLUT:** Objekttyp definieras via metadata, INTE via separat typregister

Stämmer detta? Har du ett separat typregister som borde tas bort?"

Fråga 6 - Orderkonceptet:

"Jag förstår att Orderkonceptet är systemets motor med:

- **Huvud:** Kund (från kundregistret), leveranstid (ner till timmar:minuter), leveransperioder (tidsfönster per år), periodicitet, slutdatum, beställningsreferens (fast/dynamisk), leveransadress (normalt från

objekt, override möjlig)

- **Artikellista:** Artiklar med antal (manuellt eller via metadatalogik)
 - **Metadatavillkor:** Kan styra kundnummer, kundreferens, leveranstider
 - **Generering:** Systemregel med rullande period (t.ex. 18 månader)
 - **Bekräftelseflöde:**
 1. Peka orderkoncept på objekt/kluster
 2. System matchar artikelvillkor mot objektmetadata ('haka på')
 3. Antal resolves: orderkoncept → metadatafält → utförare
 4. Uppgifter skapas per objekt, per leveransperiod
 5. Multiplikation: 1 artikel → N objekt → N uppgifter
 - **Dynamisk uppdatering:** Metadata-ändring → alla framtida ogjorda uppgifter uppdateras
- Bekräfta/korrigerar varje punkt ovan."

Fråga 7 - Uppgiftslistan/Grovplanering:

"Jag förstår att uppgiftslistan:

- Är en gemensam lista för ALLA genererade uppgifter
 - Samma format oavsett var man kommer åt den (kundvy, grovplanering)
 - **Obligatoriska kolumner:** Vad (artikel), Var (objekt/plats), När (leveranstid), Vem (uppgiftstyp)
 - Administratörer kan lägga till metadata-kolumner
 - **Tidskonsolidering:** 'Hela maj, torsdagar' → första torsdagen i maj
 - **Platskonsolidering:** 30-meters radie, men bara inom samma kund
 - Gruppering för fakturering: per objekt + per beställningsreferens
- Stämmer detta? Hur hanterar du tidskonsolideringen idag?"

B3. Förståelseverifiering - Relationer

Fråga 8: "Bekräfta/korrigerar följande relationer:

- Kund ↔ Objekt/Kluster: **Many-to-Many** (flera kunder kan dela kluster)
- Kund → Orderkoncept: **One-to-Many** (kund kan ha flera orderkoncept)
- Orderkoncept → Uppgifter: **One-to-Many** (genererar flera uppgifter)
- Orderkoncept → Artiklar: **Many-to-Many** (orderkoncept har lista av artiklar)
- Artikel → Underartiklar (struktur): **One-to-Many** (hierarki)
- Objekt → Förälder: **Many-to-One** (trädstruktur)
- Objekt → Metadatavärden: **One-to-Many**
- Artikel → Metadatavillkor: **One-to-Many** (haka-på, antal, lämna info)
- Utförare/Team → Uppgifter: koppling vid bokning
- Metadatadefinition → Metadatavärden: **One-to-Many**

Finns det relationer jag missat?"

B4. Tekniska designfrågor

Fråga 9 - Multi-tenancy:

"Hur hanteras flerföretagsstöd (multi-tenancy) i nuvarande implementation?"

- Tenant ID på alla tabeller?
- Separata scheman per företag?
- Delade vs. isolerade tabeller?

Mats specificerar att jobb inom 30m bara grupperas för SAMMA KUND. Hur säkerställs detta?"

Fråga 10 - Data som delas vs. isoleras:

“Vilka av följande entiteter kan delas mellan företag (tenants)?

- Metadatatdefinitioner?
- Artikeltyper?
- Utförandetyper?
- Ikoner?
- Eller är ALLT isolerat per tenant?”

Fråga 11 - Historik och versionering:

“Mats nämner att metadata loggar vem som uppdaterade och när. Hur hanteras:

- Historik för metadatatvärden (audit trail)?
- Versionering av orderkoncept (om kund ändrar avtal)?
- Arkivering av utförda uppgifter?
- Historiska priser vs. aktuella priser?”

Fråga 12 - Cascade-delete:

“Vilka cascade-delete beteenden finns?

- Om en kund tas bort → vad händer med orderkoncept och uppgifter?
- Om ett objekt tas bort → vad händer med metadatatvärden och uppgifter?
- Om en artikel tas bort → vad händer med orderkoncept som refererar den?
- Om ett metadatafält (definition) tas bort → vad händer med existerande värden?”

Fråga 13 - Index och prestanda:

“Vilka index finns idag? Baserat på Mats beskrivning behövs åtminstone:

- Geospatial index för lon/lat (30m radie-sökningar)
- Index på metadatafält-nummer + objektreferens (snabb metadata-lookup)
- Index på orderkoncept-kund + status (kundvy)
- Index på uppgift-leveranstid + status (grovplanering)
- Index på leveransperiod + uppgiftstyp (tidsfiltrering)
- Sammansatt index på tenant_id + alla ovanstående

Vilka av dessa finns? Vilka saknas?”

B5. Framtida skalbarhet**Fråga 14:**

“Vilka delar av nuvarande struktur kan bli problematiska vid:

- 100 000+ objekt per tenant?
- 1 000 000+ uppgifter per år?
- 50+ metadatafält per objekt?
- Komplexa ‘haka på’-villkor med 5+ kriterier per artikel?”

Fråga 15:

“Finns det risker för tight coupling som kan hindra framtida utveckling? Specifikt:

- Är Fortnox-kopplingen hårdkodad eller abstraherad bakom ett interface?
- Kan man i framtiden byta ut Fortnox mot ett annat bokföringssystem?
- Är metadatalogiken (arv, villkor) hårdkodad eller konfigurationsdriven?”

Fråga 16:

“Hur påverkas strukturen om vi senare vill lägga till:

- Mobilapp för utförare med offline-stöd?
- Realtidsspårning av utförare (GPS)?

- Integration med tredjepartssystem (t.ex. WMS, TMS)?
- AI-baserad optimering av rutter och schemaläggning?
- Kundportal med self-service?"

C. STRUKTURERAD CHECKLISTA

INSTRUKTION TILL REPLIT: Gå igenom varje tabell nedan. Bekräfta för varje:

1. ☒ Finns tabellen? (ange modellnamn i Prisma)
2. ☒ Matchar fälten Mats beskrivning?
3. ☐ ⚠ Vilka avvikelser finns?
4. ☐ ❌ Vad saknas?

C1. Kundregister (Customer)

Fält	Typ	Constraints	Status
kundnummer	String	PK, UNIQUE, NOT NULL	☐
kundnamn	String	NOT NULL	☐
prislista_id	FK	Referens till prislista	☐
fakturavillkor	String/Enum	Framtida	☐
fortnox_synk	Boolean/Timestamp	Synkstatus	☐
tenant_id	FK	Multi-tenant	☐
fria_datafält	JSON/Relation	Dynamiska fält	☐

Relationer:

- ☐ Kund ↔ Objekt/Kluster (N:M kopplingsstabell)
- ☐ Kund → Orderkoncept (1:N)
- ☐ Kund → Uppgifter (via orderkoncept)

C2. Artikelregister (Article)

Fält	Typ	Constraints	Status
artikelnummer	String	PK/UNIQUE, NOT NULL	☐
beskrivning	String	NOT NULL	☐
listpris	Decimal		☐
kostnad	Decimal		☐
produktionstid	Int/Decimal	Minuter	☐
offsettid	Int/Decimal	+/- minuter	☐
leveranstid	Int/Decimal	Logistiktid	☐
har_geo_koppling	Boolean		☐
lagerplats_geo	JSON/Coords	Sekundär geo	☐
utforandetypp_id	FK	Utförandetypp	☐
artikeltyp_id	FK	Kategorisering	☐
ar_strukturartikel	Boolean		☐
tenant_id	FK	Multi-tenant	☐

Relationer:

- ☐ Artikel → Informationsfiler (1:N) – bilder, PDF:er
 - ☐ Artikel → Underartiklar (1:N, self-referencing)
 - ☐ Artikel → Metadatavillkor-Antal (konfiguration)
 - ☐ Artikel → Metadatavillkor-LämnalInfo (1:N)
 - ☐ Artikel → Metadatavillkor-HakaPå (1:N)
 - ☐ Artikel → Utförandetypp (N:1)
 - ☐ Artikel → ExternaLeverantörer (1:N)
-

C3. Objektregister (Object)

Fält	Typ	Constraints	Status
id/löpnummer	Int/String	PK, AUTO	☐
namn	String	NOT NULL	☐
förälder_id	FK (self)	Nullable (rot-objekt)	☐
tenant_id	FK	Multi-tenant	☐

Relationer:

- ☐ Objekt → Barn-objekt (1:N, self-referencing)
- ☐ Objekt → Metadatatvärden (1:N)
- ☐ Objekt ↔ Kunder (N:M)
- ☐ Objekt → Uppgifter (1:N)

OBS: Inga typfält – ALL beskrivning via metadata!

C4. Metadatadefinition (MetadataFieldDefinition)

Fält	Typ	Constraints	Status
referensnummer	Int/String	PK, UNIQUE	☐
namn	String	NOT NULL	☐
typ	Enum	NUMERIC/ALPHANUMERIC/PHOTO/FILE/SELECTION	☐
arv_logik	Enum/JSON	Hur det ärver neråt i kluster	☐
tillåt_galleri	Boolean	Flera värden vs. senaste	☐
förälder_fält_id	FK (self)	Hierarkiska fält (kontakt)	☐
metadatatyp_id	FK	Gruppering/kategorisering	☐
tenant_id	FK	Multi-tenant	☐

Relationer:

- ☐ Definition → Underordnade definitioner (1:N, self-referencing)

- ☐ Definition → Metadatavärden (1:N)
- ☐ Definition → Metadatatyp/Grupp (N:1)

C5. Metadatavärden (MetadataValue)

Fält	Typ	Constraints	Status
id	Int	PK, AUTO	☐
definition_id	FK	NOT NULL	☐
objekt_id	FK	Nullable	☐
värde_text	String	Nullable	☐
värde_nummer	Decimal	Nullable	☐
värde_foto	String/URL	Nullable	☐
värde_fil	String/URL	Nullable	☐
uppdaterad_av	FK (User)	Logg	☐
uppdaterad_tidpunkt	Timestamp	Logg	☐
tenant_id	FK	Multi-tenant	☐

C6. Orderkoncept (OrderConcept)

Fält	Typ	Constraints	Status
id	Int	PK, AUTO	☐
kund_id	FK	NOT NULL	☐
leveranstid_start	Time		☐
leveranstid_slut	Time		☐
leveransperioder	JSON/Relation	Tidsfönster	☐
periodicitet	String/Enum	t.ex. varannan månad	☐
startdatum	Date		☐
slutdatum	Date	Nullable	☐
beställningsreferens	String	Fast eller metadata-ref	☐
leveransadress_manuell	String	Nullable (override)	☐
status	Enum	AKTIV/UTFÖRD/ARKIVERAD	☐
mall_id	FK	Nullable (skapad från mall)	☐
tenant_id	FK	Multi-tenant	☐

Relationer:

- ☐ Orderkoncept → Kund (N:1)
 - ☐ Orderkoncept → Artikelrader (1:N – kopplingsstabell med antal)
 - ☐ Orderkoncept → Leveransperioder (1:N)
 - ☐ Orderkoncept → Metadatavillkor (1:N)
 - ☐ Orderkoncept → Genererade uppgifter (1:N)
 - ☐ Orderkoncept → Objekt/Kluster (koppling vid bekräftelse)
-

C7. Uppgift / Task

Fält	Typ	Constraints	Status
id	Int	PK, AUTO	☐
orderkoncept_id	FK	NOT NULL	☐
artikel_id	FK	NOT NULL	☐
objekt_id	FK	Nullable	☐
antal	Decimal	NOT NULL	☐
leveranstid_start	DateTime		☐
leveranstid_slut	DateTime		☐
status	Enum	PLANERAD/TILLDELAD/UTFÖRD/FAKTURERAD	☐
utförare_id	FK	Nullable	☐
team_id	FK	Nullable	☐
beställningsreferens	String	Från orderkoncept/ metadata	☐
jobb_grupp_id	FK	30m-gruppering	☐
tenant_id	FK	Multi-tenant	☐

C8. Utförandety (PerformerType)

Fält	Typ	Constraints	Status
id	Int	PK, AUTO	☐
namn	String	NOT NULL, UNIQUE	☐
tenant_id	FK	Multi-tenant	☐

C9. Utförare (Performer)

Fält	Typ	Constraints	Status
id	Int	PK, AUTO	☐
namn	String	NOT NULL	☐
typ	Enum	PERSON/UTRUSTNING	☐
utforandetyd_id	FK		☐
arbetstid_schema	JSON/Relation	40h/vecka, tillgänglighet	☐
tenant_id	FK	Multi-tenant	☐

Relationer:

- ☐ Utförare → Team (N:M kopplingsstabel)
- ☐ Utförare → Uppgifter (via bokning)

C10. Team

Fält	Typ	Constraints	Status
id	Int	PK, AUTO	☐
namn	String		☐
tillgänglighet	JSON/Relation	Tidschema	☐
tenant_id	FK	Multi-tenant	☐

C11. Artikeltyp (ArticleType)

Fält	Typ	Constraints	Status
id	Int	PK, AUTO	☐
namn	String	UNIQUE	☐
tenant_id	FK		☐

C12. Metadatatyp (MetadataType)

Fält	Typ	Constraints	Status
id	Int	PK, AUTO	☐
namn	String	UNIQUE	☐
gruppering	String	t.ex. kvalitetskontroll	☐
tenant_id	FK		☐

C13. Ikonregister (Icon)

Fält	Typ	Constraints	Status
id	Int	PK, AUTO	☐
namn	String		☐
fil/url	String		☐
tenant_id	FK		☐

D. SCHEMA-JÄMFÖRELSE

INSTRUKTION TILL REPLIT:

D1. Visa nuvarande schema

“Visa hela ditt nuvarande Prisma-schema (eller motsvarande databasschema) för Traivo. Inkludera:

- Alla modeller/tabeller
- Alla fält med typer
- Alla relationer
- Alla enum-definitioner
- Alla index och constraints”

D2. Gör en diff

“Jämför Mats beskrivning (sammanfattad ovan) med ditt nuvarande schema och identifiera:

Saknas helt:

- Vilka tabeller/modeller från Mats beskrivning finns INTE i nuvarande schema?
- Vilka relationer saknas?
- Vilka fält saknas?

Implementerat annorlunda:

- Vilka koncept har du löst på ett annat sätt?

- Finns det fall där du har EN tabell där Mats beskriver TVÅ (eller vice versa)?
- Hanterar du metadata annorlunda (t.ex. JSON-fält istället för separata tabeller)?

Kan orsaka framtida problem:

- Finns det tight coupling som kan göra det svårt att ändra?
- Finns det N+1 query-risker i nuvarande relationsstruktur?
- Finns det denormalisering som kan leda till inkonsistens?"

D3. Avvikelserapport

"Producera en tabell i format:

#	Mats beskrivning	Nuvarande implementation	Avvikelse	Risk	Rekom-mendation
1	...	...	...	Hög/Medium/Låg	...
"					

E. MIGRATIONSSTRATEGI

INSTRUKTION TILL REPLIT: Om avvikelser hittades i D2, besvara följande:

E1. Påverkan på befintlig data

"För varje avvikelse som kräver ändring:

- Finns det befintlig data som påverkas?
- Hur mycket data är det (uppskattning)?
- Kan data transformeras automatiskt eller krävs manuell hantering?"

E2. Migrationssteg

"Beskriv en steg-för-steg migrationsplan:

1. Vilka schemaändringar behövs?
2. I vilken ordning ska de göras (beroenden)?
3. Krävs datamigreringsscript?
4. Hur lång driftstopp krävs (om något)?"

E3. Breaking changes

"Identifiera alla breaking changes:

- Vilka API-endpoints påverkas?
- Vilka frontend-komponenter måste uppdateras?
- Finns det externa integrationer (Fortnox) som påverkas?"

E4. Rollback-plan

"Om migreringen misslyckas:

- Kan vi rulla tillbaka?"

- Vad krävs för rollback?
- Finns det backup-strategier?"

E5. Testning av migrering

"Hur testar vi migreringen säkert?

- Kan vi köra migreringen i staging först?
- Hur verifierar vi att all data överfördes korrekt?
- Hur testar vi att befintlig funktionalitet fortfarande fungerar?"

F. PRIORITERADE FRÅGOR FÖR OMEDELBAR KLARGÖRING

Följande frågor är **MEST KRITISKA** och bör besvaras FÖRST:

● Topp 5 - Måste besvaras innan ALLT annat

1. **Metadata-arkitektur:** "Har du TVÅ separata tabeller/modeller för metadata (definition + värden) eller EN? Om en – hur skiljer du på struktur och data?"
2. **Orderkoncept → Uppgift-generering:** "Beskriv exakt hur ditt system idag genererar uppgifter från ett orderkoncept. Matchar det Mats flöde: orderkoncept → matcha artikelvillkor mot objektmetadata → skapa uppgift per objekt per leveransperiod?"
3. **'Haka på'-logiken:** "Hur implementerar du villkoret att en artikel ska 'haka fast' på ett objekt baserat på metadataavillkor? Finns det en regelmotor eller är det hårdkodat?"
4. **Dynamisk uppdatering:** "Om en utförare uppdaterar ett metadatavärde (t.ex. antal kärl 10→12), uppdateras då ALLA framtida ogjorda uppgifter automatiskt? Hur triggas detta?"
5. **Multi-tenancy:** "Har ALLA tabeller tenant_id? Filtrerar ALLA queries på tenant? Finns det risk för dataläckage mellan företag?"

G. SVARSFORMAT

INSTRUKTION TILL REPLIT: Strukturera dina svar enligt detta format:

```
### Fråga [nummer]
**Status:** ☒ Stämmer / ☐ Delvis / ☐ Avviker
**Nuvarande implementation:** [beskriv]
**Avvikelse:** [beskriv om relevant]
**Risk:** Hög / Medium / Låg
**Åtgärd krävs:** Ja / Nej
**Kommentar:** [detaljer]
```